EXPERNETIC · TALENT ASSESSMENT PROGRAM

Edinburgh Airbnb Market Intelligence

Data Engineering & Analytics Technical Assignment

Candidate	Noor Mohammed Mohammed Fiham
Role applied for	Data Engineer Intern
City analyzed	Edinburgh, Scotland, United Kingdom (single-city scope)
Data source	Inside Airbnb — snapshot dated 21 September 2025
Submission date	30 June 2026

Table of Contents

01 Executive Summary

This report covers a full analysis of Edinburgh's short-term rental market, built on the Inside Airbnb dataset (21 September 2025 snapshot: 4,936 listings, 1.8 million calendar-day records, 559,087 reviews). I took the project through the whole pipeline myself: ingestion, cleaning, dimensional modeling, exploratory analysis, formal statistics, and a machine learning experiment with explainability analysis on top.

£158	3,037	79.4%	R^2=0.49
MEDIAN NIGHTLY PRICE	UNIQUE HOSTS	SINGLE-LISTING HOSTS	BEST PRICE MODEL FIT

Key Findings

- Entire-home listings cost noticeably more than private rooms. Median £186 vs. £87. The effect size confirms this isn't just a sample-size artifact (rank-biserial r=0.72, a large effect).
- Edinburgh's hosts split into two very different groups. Most (79.4%) run a single listing, but the top 10% control 38% of all supply. There's a real professional-operator segment here alongside the casual hosts.
- Review scores are heavily inflated. 91.3% of listings sit at 4.5 stars or above, which means the rating itself doesn't tell you much about which listings are actually better.
- Location matters, but it's not the whole story. Neighbourhood alone explains about 14% of price variance. Add in property characteristics and that climbs to 41–49%, leaving roughly half still unaccounted for.
- Weekends are genuinely busier. Friday/Saturday nights show lower availability than weekdays (44.0% vs. 45.9%) — but the effect is small. I'd rather say that plainly than let a tiny p-value imply something bigger than it is.
- The calendar data's price field turned out to be completely empty in this snapshot. I checked this wasn't a parsing mistake on my end, confirmed it was a real gap in the source data, and worked around it using Inside Airbnb's own precomputed occupancy and revenue fields instead.
- A price-prediction model gets to a usable but imperfect fit (around 33% mean error). The most interesting result from it: parking predicts *lower* price, not higher, because parking availability tracks with being outside Edinburgh's dense, expensive centre rather than being a desirable feature on its own.

A Note on How This Was Built

I want to be upfront about something: a fair number of the charts and numbers in this report went through a correction step before they ended up here. I'd build something, look at it again, realize it was either wrong or somewhat misleading, and fix it. Rather than quietly clean that up and present only the final polished version, I've left those corrections visible throughout the report and in the project's decision log. I think that process says more about how I work than a spotless first draft would.

02 Objectives & Scope

What I Was Trying to Achieve

Expernetic's brief evaluates seven things: data engineering fundamentals, analytical thinking, data science and ML, AI/LLM literacy, communication, creativity, and professionalism. The brief is also explicit that it's scoped beyond what one person can finish in a week, and says so directly — depth on a few sections beats shallow coverage of everything. I took that at face value and planned around it rather than trying to tick every box.

Why Edinburgh, and Why Just One City

I picked Edinburgh and stuck to a single-city scope. The assignment's own city selection guide frames a single city as the right call when you want depth, code quality, and a real narrative, and saves multi-city work for demonstrating pipeline scalability specifically. Three things made Edinburgh a good fit beyond just "it's on the list":

- **Size.** Around 4,900 listings and 1.8 million calendar rows is enough to run real statistics and segmentation on, but small enough that I could iterate quickly without needing distributed processing. Cities like London or New York run past 100,000 listings in this dataset — great for showing off scale, bad for a five-day window where iteration speed matters more.
- **A genuine story to tell.** Edinburgh's festival calendar (the Fringe, Hogmanay) and its active short-term-let licensing regime both show up directly in the data, which meant the seasonal and regulatory angles in this report came from the data rather than being bolted on afterward.
- **Recent data.** The snapshot I used was reasonably fresh at the time I started.

How I Prioritized

I mapped my available time against the rubric's own stated weights rather than guessing. Problem Solving sits at 30%, Data Engineering Quality at 25%, and Statistical Thinking and Analytical Storytelling at 20% each — so that's roughly where the bulk of the five days went.

Day	Focus	Assignment Section(s)	Why
1	Dataset familiarization, ingestion & profiling	2, 3.1	Everything downstream depends on this being solid
2	Cleaning, standardization, dimensional modeling	3.2–3.4	Highest single rubric weight (25%)
3	Exploratory data analysis	4	Drives the storytelling score directly
4	Statistical hypothesis testing & regression	5	Independently weighted at 20%
5	ML price prediction with SHAP, final report	6.1, 12	Best use of remaining time given existing regression work; the report is what actually gets read

WHAT I LEFT OUT

I didn't touch Section 3.6 (cloud-native/production topics), Section 5.4 (multi-city comparisons), most of Section 7 (NLP/LLM/agentic experimentation), or Section 8 (the open innovation challenge). That's a deliberate trade-off, not something I ran out of time for by accident — see Sections 13 and 14 for what I'd actually do with these given more time.

03 Dataset Overview

Source & Files

Everything here comes from Inside Airbnb, an independent project that scrapes publicly listed Airbnb data. I used five files: `listings.csv` (4,936 rows, 79 columns), `calendar.csv` (1,801,640 rows), `reviews.csv` (559,087 rows), `neighbourhoods.csv` (111 rows), and `neighbourhoods.geojson` (boundary polygons for those same 111 neighbourhoods).

> **FIRST DATA QUALITY SURPRISE**
>
> A simple `wc -l listings.csv` reports 10,165 lines, more than double the actual row count. The reason: several text fields (descriptions, neighbourhood overviews, the amenities list) contain line breaks inside quoted CSV cells, which a naive line counter doesn't understand. I confirmed the true count (4,936) using pandas, which respects CSV quoting properly, and double-checked it against the `id` column having zero nulls and exactly 4,936 unique values. Small thing, but worth knowing before you trust any quick command-line check on this kind of file.

How the Tables Relate

`listings.id` is the primary key. There's no separate hosts table in the source data, which surprised me at first — host attributes like name, join date, and superhost status are all just columns inside `listings.csv`, so a proper hosts dimension had to be derived rather than pulled directly (more on that in Section 5). `calendar.listing_id` and `reviews.listing_id` both join back to listings, and geography joins through `neighbourhood_cleansed`, not the raw `neighbourhood` field, which I'll explain below.

Field Decisions Worth Knowing About

Field	Issue	What I did
`neighbourhood` (raw)	43.8% null, and the values that exist are just host-written marketing text, not real neighbourhood names	Ignored it. Used `neighbourhood_cleansed` instead, which lines up exactly with the reference table
`calendar.price` / `adjusted_price`	100% null across all 1.8 million rows, confirmed it wasn't a parsing mistake	Used `listings.csv`'s precomputed occupancy and revenue fields instead; calendar stayed useful for availability patterns
`license`	75.4% null	Treated null as "not disclosed," not "unlicensed." Didn't want to imply anything about compliance I couldn't actually support
Price exactly $9,999 (10 listings)	Shows up across totally unrelated property types with no logical connection to price	Flagged these as placeholder values and excluded them from price analysis, but kept them in the dataset rather than deleting anything
92 listings sharing host + exact coordinates	Looked like duplicates at first glance	Checked it. Turned out to be a real commercial operation (one host running 16 individually listed hostel rooms at one address). Kept these and used the pattern later for host segmentation

The full schema notes and the complete assumptions log live in `reports/01_dataset_familiarization.md` and `reports/00_assumptions_and_decisions.md` if you want more detail than fits here.

Limits of This Dataset

Worth saying plainly: this is scraped data, not Airbnb's own internal records, so it's an approximation. Inside Airbnb's own documentation puts listing-count accuracy at roughly within 10–20%. It's also a single snapshot in time — it won't catch anything added or removed outside that one date. Throughout this report I use review counts as a stand-in for booking demand, since real booking numbers aren't public. That's a known limitation of working with this kind of data generally, not something specific to how I handled it.

04 Methodology

Overall Approach

I followed a fairly standard lifecycle: ingest and profile the raw data, clean it into something analysis-ready, build it into a proper queryable structure, then layer EDA, statistics, and ML on top of that same structure. The dimensional model from Section 5 isn't a one-off — every chart and every statistical test in this report queries it directly, so nothing downstream is working from its own separate copy of the data.

Why These Tools

Task	Tool	Reasoning
Small, wide tables (listings, neighbourhoods)	pandas	Most of the work here is row-level string and category cleaning, which pandas handles well at this scale
Large tables (calendar, reviews)	DuckDB	Far faster for aggregation at 1.8M+ rows; also doubles as the home for the dimensional model, so there's no extra export step
Statistical testing	scipy, statsmodels	Standard, well-documented implementations with clear assumption requirements
Machine learning	scikit-learn, XGBoost, SHAP	Covers all three required model families plus explainability
Geospatial	GeoPandas	The standard choice for joining tabular stats onto GeoJSON boundaries

How I Picked Statistical Tests

I didn't default to the most familiar test for a given question. Price turned out to be heavily right-skewed everywhere I looked (raw skew of 14.9), and that single fact shaped most of the methodology choices in Section 7: non-parametric tests instead of t-tests or ANOVA wherever skew or unequal variance was confirmed, log-transforming price before running regression, and reporting more than one effect-size measure in places where they actually disagreed with each other (see Section 7.1 for a concrete example of why that matters).

Reproducibility

Everything is real Python modules in `src/`, not just notebook cells that only run in order if you're careful. Each one runs independently against the actual dataset. There's also a single combined, fully-executed notebook (`notebooks/02_eda_and_statistics.ipynb`) that walks through the EDA, statistics, and ML sections using the exact same functions as this report, so the two can't drift apart from each other over time.

05 Engineering Approach

Pipeline Structure

The pipeline is five sequential Python modules in `src/`: `ingest.py` handles profiling and validation, `clean.py` does the cleaning and standardization, `model.py` builds the dimensional model, `stats_analysis.py` and `regression_analysis.py` run the statistics, and `ml_price_prediction.py` handles price prediction and SHAP. Each one reads what the previous stage produced rather than redoing work, and `config.py` keeps city configuration in one place so this could point at a different Inside Airbnb city by editing a single file, not by hunting through logic scattered across modules.

The Dimensional Model

I built a star schema in DuckDB. **fact_listing** is the centre of it — one row per listing, 4,936 rows — joined to three dimensions: **dim_host** (3,037 unique hosts, derived by deduplicating host attributes out of `listings.csv` since there's no source hosts table), **dim_neighbourhood** (111 rows), and **dim_date** (365 rows, with weekend, season, and festival-month flags already computed so I wasn't recalculating date logic in every query). `calendar_raw` and `reviews_raw` stay as their own large tables rather than getting restructured into more fact tables, since they're already at a sensible grain and join cleanly to `fact_listing` through `listing_id`.

> **CHECKING IT ACTUALLY WORKED**
>
> Before trusting this model for anything, I ran three test queries against it: top neighbourhoods by price, superhost vs. non-superhost rating, and top hosts by listing count. Zero unmatched foreign keys across all 4,936 rows on both join keys, which told me the joins were solid before I built anything else on top of them.

Some Decisions Along the Way

The full log (`reports/00_assumptions_and_decisions.md`) has 24 dated entries. A few worth calling out here, mostly because each one involved getting something wrong first and then catching it:

Decision	What happened
Calendar pricing gap	Confirmed the price field in `calendar.csv` is entirely null, not a parsing bug on my end. Used the precomputed listings-level fields instead of trying to fabricate a flat-rate estimate
Boolean column type	Caught during validation: an early cleaning pass left a boolean column as a generic object type instead of pandas' proper nullable boolean. Fixed and re-checked
Small-sample neighbourhoods	A first version of the price map ranked one neighbourhood as "most expensive" based on just 2 listings. Fixed by excluding anything under 10 listings from price rankings, applied consistently in both the maps and the statistical test later
VIF calculation bug	A multicollinearity check initially threw up false warnings (15.2, 9.1) because I'd dropped the regression constant before running the VIF function. Caught it by checking the raw correlations didn't support numbers that high
Actually running everything	Every script and notebook in this project got executed and checked for errors, not just written and assumed to work. I caught a missing import this way during final assembly

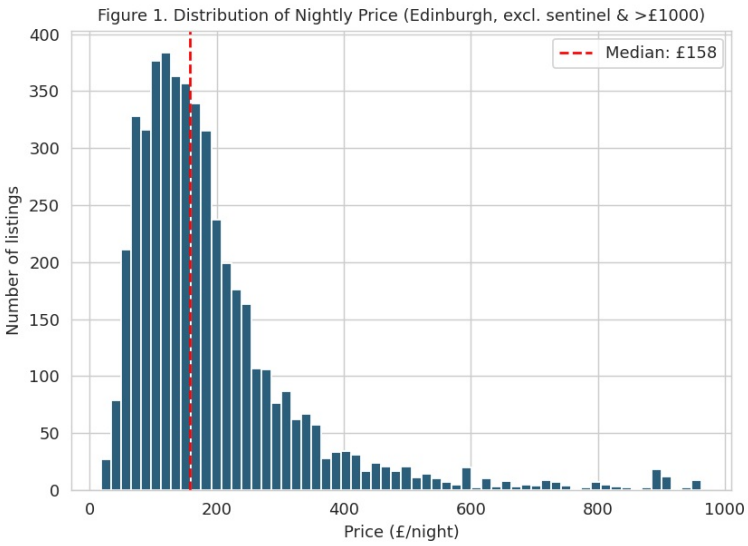
What I Didn't Build

Section 3.6 (cloud-native topics) wasn't part of this submission. For what it's worth: scaling this to 50+ cities would mean partitioning the calendar table by city and date, a real orchestration layer instead of running scripts in sequence, and proper lineage tracking on when each table was last touched. These are concrete next steps, listed in Section 14, not half-built scaffolding I'm pretending isn't there.

06 EDA Findings

Price-based analysis below excludes the 10 listings flagged as placeholder-priced at exactly £9,999 and 11 listings with no price at all, leaving 4,915 of 4,936. Neighbourhood-level numbers exclude the 22 of 111 neighbourhoods with fewer than 10 listings. More on why that matters below.

6.1 Price by Room Type



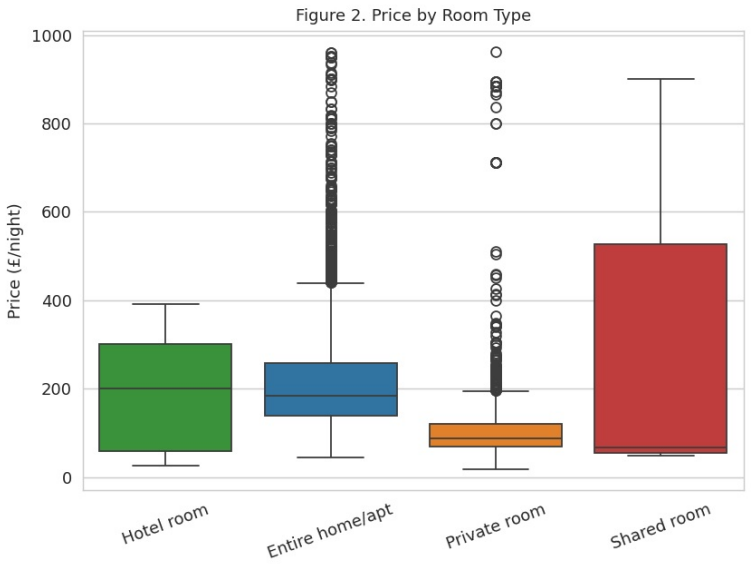
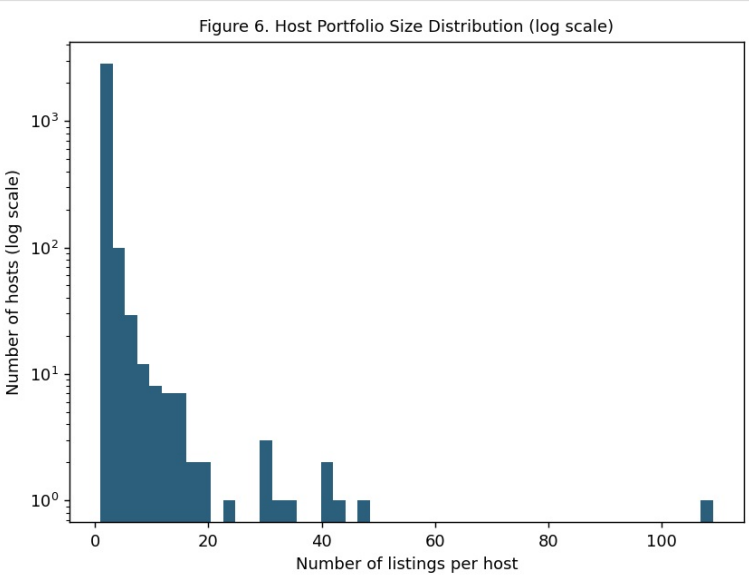


Figure 1–2. Distribution of nightly price overall (left) and by room type (right).

Median price across Edinburgh is **£158/night**. Entire homes and hotel rooms sit at the top, private rooms cost roughly half what an entire home does. If you're benchmarking new supply, use the median rather than the mean (£280–300). The mean gets pulled up by a relatively small number of expensive listings.

> **ONE THING THAT LOOKED ODD AT FIRST**
>
> The boxplot shows some "Shared room" listings priced as high as £900/night, which doesn't make sense for what should be the cheapest category. Turns out these are hostel listings where the price is for booking the entire dorm room, not one bed. A handful of operators do this. Worth knowing before you read too much into the shared-room numbers.

6.2 Who Owns the Listings



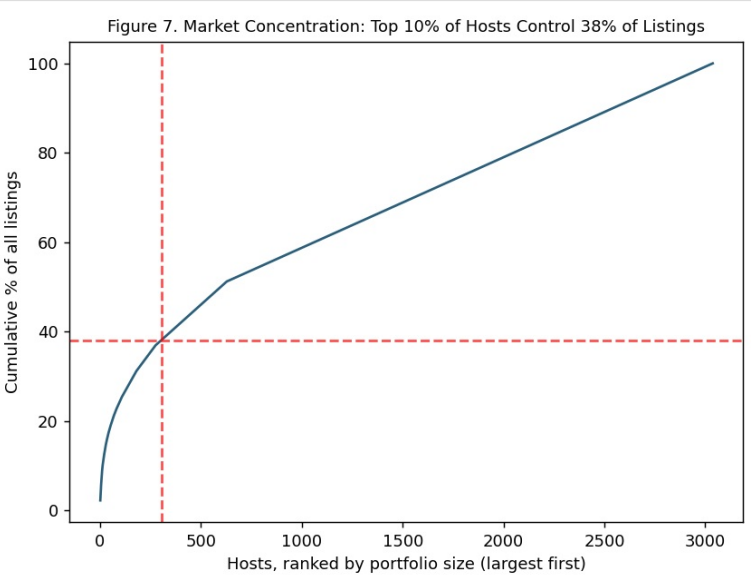


Figure 6–7. Host portfolio size distribution (log scale) and market concentration curve.

79.4% of the 3,037 hosts run exactly one listing. But the top 10% (303 hosts) control 38% of everything — one host alone has 110 listings, almost certainly run as a business rather than a hobby. This isn't purely a casual-host market, and host support or platform policy probably needs to account for both ends of that spread rather than assuming one typical host profile.

6.3 Where the Money Is

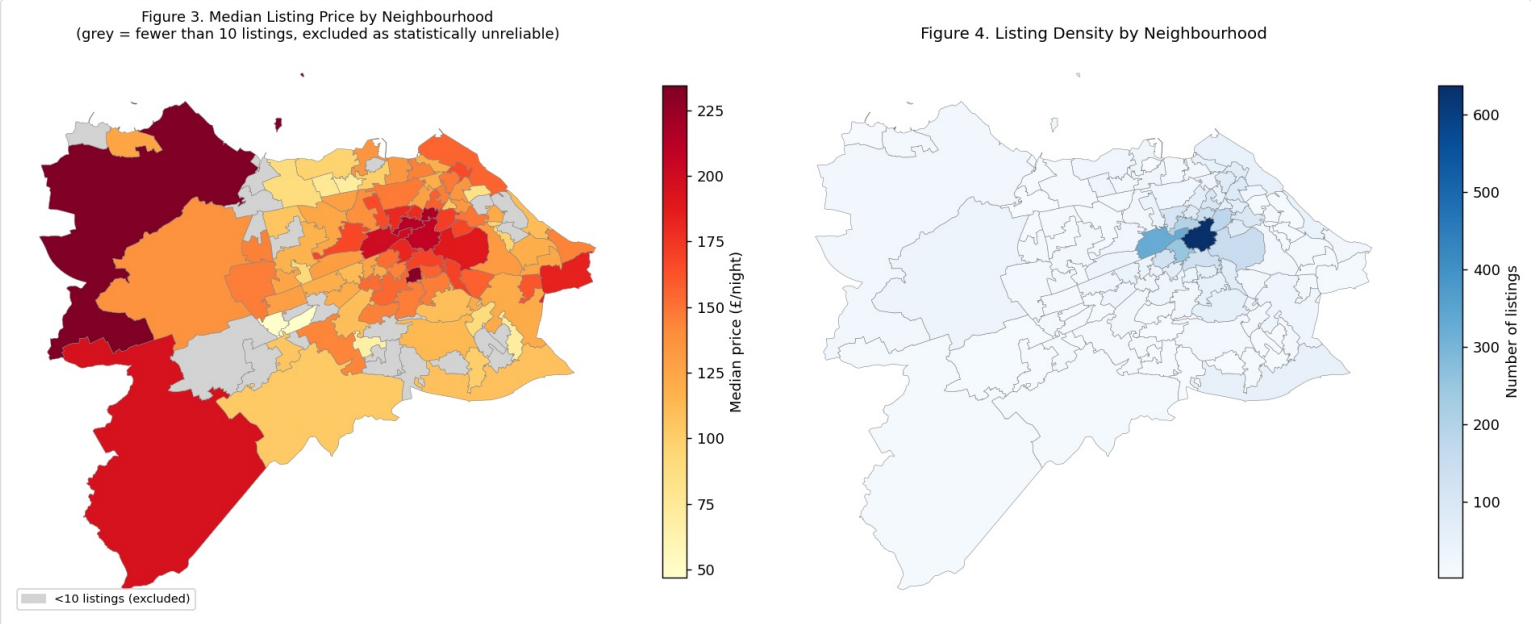


Figure 3–4. Median price by neighbourhood (left, grey = excluded for small sample) and listing density (right).

Both price and listing density cluster around central Edinburgh, which tracks with proximity to the Old Town, New Town, and the main tourist draws. One result that holds up even after filtering out small samples: **Dalmeny, Kirkliston and Newbridge**, near the airport, has the highest reliable median price outside the centre (£234.5), and listings there accommodate 4.4 guests on average against 3.66 city-wide. Reads more like larger group-oriented properties than a pure location premium.

> **I HAD TO FIX THIS MAP ONCE ALREADY**
>
> The first version of this ranked a different neighbourhood as Edinburgh's most expensive, based on a median of 2 listings. That's not a finding, that's noise. I added a 10-listing minimum before any neighbourhood shows up in a price ranking (shown grey on the map otherwise), and kept that threshold consistent everywhere in this report, including the formal statistical test in Section 7.

6.4 Seasonality

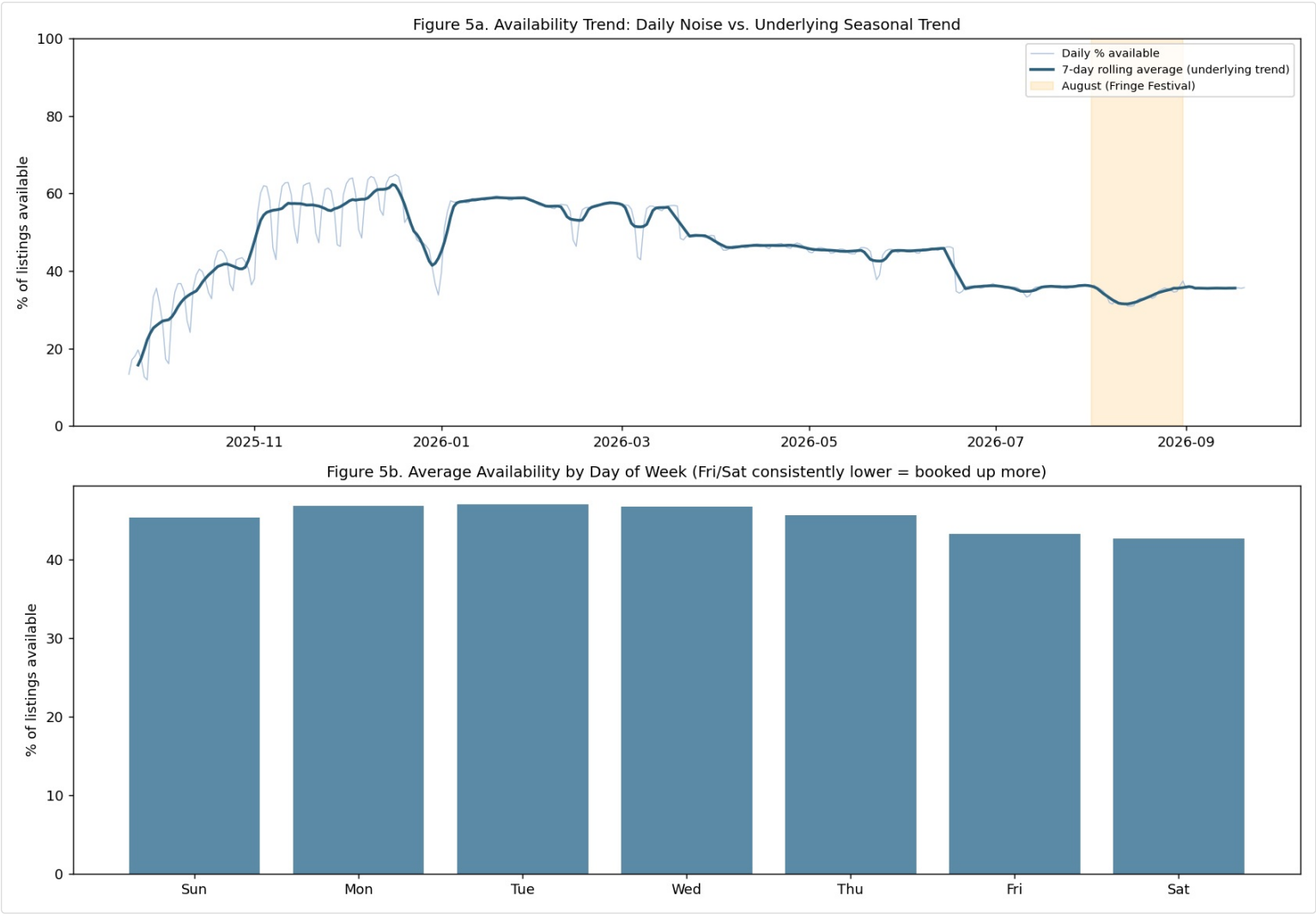


Figure 5. Availability trend (daily values and 7-day rolling average) and average availability by day of week.

A REAL LIMITATION HERE

The calendar's price field is empty across the board in this snapshot, so this section is about availability, not day-by-day price.

Two patterns showed up, and they're worth separating. There's a weekly cycle: Friday and Saturday nights are consistently less available than weekdays, every single week across the 12-month window. That's a genuine, year-round demand signal, and I test it formally as H5 in Section 7. There's also a slower seasonal trend underneath it: availability climbs toward a plateau around November to February, then drops steadily from spring through to its low point in August.

WHAT I ALMOST SAID, BUT COULDN'T ACTUALLY BACK UP

August does have the lowest availability of the year, and it's tempting to say "the Fringe Festival causes this." But looking closer, August is really just the tail end of a decline that's been running since spring. The more accurate read is that festival demand piles onto a trend that was already heading that way. My first version of this chart was a single line and made the festival look like the whole story — it wasn't, so I rebuilt it as two panels to show both patterns honestly.

6.5 Reviews

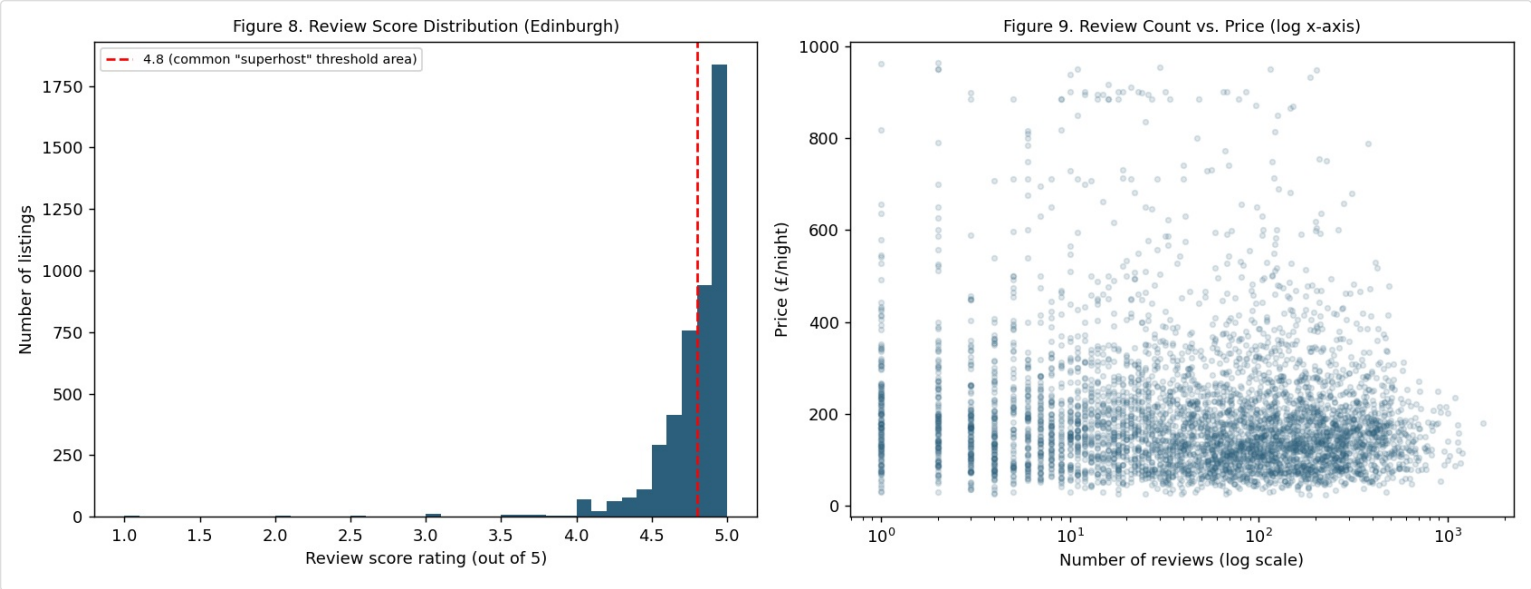


Figure 8–9. Review score distribution (left) and review count vs. price (right).

91.3% of listings score 4.5 or higher out of 5, and only 1.2% fall below 4.0. A scale that's supposed to run from 1 to 5 is, in practice, squeezed into a narrow band near the top for almost everyone. Raw review score isn't a great way to tell a good listing from a great one here. Review count, recency, and the sub-category scores probably carry more useful signal on their own.

I also didn't find much of a relationship between review count and price — popular listings (by review volume, used here as a demand proxy) span the whole price range. That's part of why I moved to regression in Section 7.3 instead of looking at price and reviews as a simple pair.

07 Statistical Findings

All tests use α=0.05, and I've reported an effect size next to every p-value. The H5 result below is a good demonstration of why that second number matters — an extremely small p-value sitting right next to an effect size that's basically negligible.

7.1 Hypothesis Tests

H1 — Entire homes cost more than private rooms

Price is strongly right-skewed in both groups (Shapiro-Wilk rejects normality at p<0.001 for each), so I used Mann-Whitney U instead of a t-test. Result: median £186 for entire homes vs. £87 for private rooms, p<0.001.

> **WHY I REPORTED TWO EFFECT SIZES HERE**
>
> Cohen's d comes out to 0.18, which reads as "negligible." Rank-biserial r comes out to 0.72, "large," for the exact same comparison. They disagree because Cohen's d depends on standard deviation, and entire-home prices have a huge spread (around £1,310) driven by a handful of luxury outliers, which shrinks d even though the median gap is obviously large. Rank-biserial r doesn't have that problem since it works off ranks instead of means. I'm reporting both rather than just quoting whichever one sounds better. My actual read: this is a real, substantial price premium for entire homes, and r is the more trustworthy number here.

H2 — Superhosts get better reviews

Same skew, same test. Median 4.90 for superhosts vs. 4.75 for everyone else, p<0.001, Cohen's d=0.69 (a medium effect). This is real and not just a function of sample size, but it's happening entirely within the compressed top of the rating scale, so in absolute terms the gap is under half a star. Superhost status is a genuine signal, just not one I'd lean on by itself.

H3 — More reviews means a different price

Median £160 for listings with more than 10 reviews vs. £155 for fewer, p<0.001, but rank-biserial r=−0.14 and Cohen's d=−0.11, both small to negligible. Statistically detectable, sure, but not practically meaningful. This lines up with what I saw in Section 6.5: probably just older listings accumulating more reviews over time, not price itself driving review counts in either direction.

H4 — Neighbourhoods differ in price

Price is right-skewed within every neighbourhood group, and Levene's test shows unequal variance across them too, so I used Kruskal-Wallis rather than a standard ANOVA. Tested across the 89 neighbourhoods with at least 10 listings: H=749.7, p<0.001, eta-squared=0.141. So neighbourhood explains around 14% of price variance by itself — real, but not the dominant factor (the regression in 7.3 puts this in context).

H5 — Weekends differ from weekdays

> **HAD TO SUBSTITUTE HERE**
>
> Calendar price is empty in this snapshot, so I couldn't test weekend vs. weekday *pricing* the way the hypothesis literally asks. I used availability instead, as the closest thing I could actually test.

Chi-square test across all 1,801,640 calendar-day rows: weekend availability 44.0% vs. weekday 45.9%, χ^2=535.3, p<0.001, but Cramer's V=0.017.

> **SAYING THIS PLAINLY RATHER THAN DRESSING IT UP**
>
> At 1.8 million observations, even a 2-point gap is going to come back "significant." That's exactly the trap effect-size reporting exists to catch. The honest version: the weekend effect is real and shows up every single week, but it's small. I wouldn't tell a host to expect a dramatic weekend booking bump from this alone, and I can't say anything about a weekend price premium given the calendar gap.

7.2 Confidence Intervals

Room type	n	Mean price	95% CI (mean)	Median	95% CI (median, bootstrap)
Entire home/apt	3,535	£337.18	[£293.99, £380.36]	£186	[£183, £189]
Hotel room	21	£191.19	[£131.80, £250.58]	£200	[£89, £289]
Private room	1,340	£133.36	[£117.64, £149.07]	£87	[£85, £90]
Shared room	19	£285.95	[£103.89, £468.00]	£68	[£55, £155]

Look at the Shared room row: the mean and median basically tell opposite stories, and the median's confidence interval is wide relative to the value itself. That's the small sample size (n=19) and the hostel pricing quirk from 6.1 showing up again. The width of that interval is itself a useful signal — it's telling you not to treat this number as precise.

7.3 What Actually Drives Price

I ran an OLS regression of log(price) on accommodates, bedrooms, bathrooms, minimum nights, review score, superhost status, and room type. Log, not raw price, because raw price skew (14.9) badly violates OLS's assumptions about residuals; log-price skew (1.5) is a real improvement, if not a perfect one. $R^2=0.414$ — the model accounts for about 41% of log-price variance.

Predictor	Coefficient	What it means
accommodates	+0.131	Each extra guest of capacity adds about 13% to price in log terms
bathrooms	+0.088	Significant, positive
bedrooms	+0.041	Significant, positive
review_scores_rating	+0.147	Significant, positive
host_is_superhost	+0.011 (n.s., p=0.533)	Not significant once you control for property characteristics
room_Private room	−0.445	Significantly cheaper than the entire-home reference category

THIS LOOKS LIKE IT CONTRADICTS H2. IT DOESN'T.

Superhost status isn't significant in this regression, which seems to clash with H2's finding that superhosts get better reviews. It doesn't actually clash. H2 is about review scores, this regression is about price once you've already accounted for property size and type. Both can be true: a superhost might genuinely earn better reviews without charging any more for an equivalent property. I'd read this as superhost status being more of a trust signal than a pricing lever.

I GOT THE MULTICOLLINEARITY CHECK WRONG THE FIRST TIME

My first VIF pass flagged accommodates at 15.2 and review score at 9.1, both suggesting serious multicollinearity. That was a bug. I'd dropped the regression's constant term before calling statsmodels' VIF function, which actually needs the constant present to compute the other columns correctly. I caught it because the raw pairwise correlations between predictors were all weak (under 0.33), which didn't square with VIF values that high. Corrected VIFs are all under 4. No real multicollinearity problem here.

Worth being honest about the limits: residuals are still right-skewed and heavy-tailed even after the log transform, so a handful of outlier listings are probably still pulling on the fit. And $R^2=0.414$ means well over half of price variance is coming from something this model doesn't capture — exact location, photo quality, how responsive the host is, things like that. I pick this thread back up in Section 8 with a model that adds amenity data on top of these same features.

08 Data Science Experiments — Price Prediction

How I Framed This

Target variable: nightly price, log-transformed for the same reason as the Section 7.3 regression (raw skew 14.9, log skew 1.5). For success criteria I used MAE and RMSE on log-price to compare models, plus MAE and MAPE back in £ terms since those are easier to actually reason about. Everything is computed from 5-fold cross-validated out-of-fold predictions, not how well the model fits the data it was trained on. I wanted an honest read on how this would do on listings it hasn't seen.

Features

I built on the same property features from the Section 7.3 regression and added:

- **10 amenity flags**, pulled out of the raw `amenities` text field (a Python-list-style string per listing, and messy enough that I had to match on substrings rather than exact text — "Free washer – In unit" and "Washer" needed to count as the same thing). I picked amenities with real variance in how common they are (20–60% of listings), since something like Wifi (present on 89.8%) doesn't tell a model much about why one listing costs more than another.
- **amenity_count**, just the total number of amenities listed, as a rough stand-in for how much effort went into the listing.
- **Two interaction terms**: accommodates times bathrooms, since capacity and bathroom count probably compound rather than just add up. A 6-person listing with 1 bathroom is a different product from a 6-person listing with 3. The second is private room times has-private-entrance, on the theory that a private entrance matters more when you're renting a room than when you've got the whole place anyway.

Final feature matrix: 4,909 rows, 22 features.

Comparing Models

Model	MAE (log)	RMSE (log)	MAE (£)	MAPE
Ridge Regression	0.348	0.543	£147.29	35.1%
Random Forest	**0.326**	0.511	£143.57	**33.0%**
XGBoost	0.329	**0.505**	£144.72	33.5%

Random Forest and XGBoost both beat Ridge by a modest margin, which makes sense since they can pick up non-linear patterns a straight linear model can't. The improvement over the Section 7.3 regression is real (R^2 goes from 0.41 to about 0.49) but it's incremental, not a huge leap. I went with Random Forest for the residual and SHAP work that follows. It has a slightly better MAE, though honestly the margin over XGBoost is close enough that either would have been a defensible choice.

Where the Model Struggles

Room type	Mean absolute % error
Hotel room	42.5%
Private room	52.7%
Entire home/apt	69.9%
Shared room	102.2%

THAT 102% ERROR ISN'T AS ALARMING AS IT SOUNDS

There are only 19 "Shared room" listings total in this dataset, which means roughly 3 or 4 per cross-validation fold, not enough for any model to learn a reliable pattern from, especially given the hostel whole-room pricing quirk from Section 6.1. I checked: dropping Shared Room entirely barely moves overall MAPE (still 33.0%). So this is a small, explainable issue, not a sign the model is broken everywhere.

Price bucket	n	Mean absolute % error
<£100	1,028	48.7%
£100–200	2,220	27.0%
£200–300	905	23.8%
£300–500	497	33.3%
£500+	259	53.2%

There's a clear U-shape here: the model is most accurate in the £100–300 range and gets worse at both ends. That's not surprising. Both ends are rarer in the data, and probably priced by things this feature set just doesn't capture (budget-hostel pricing conventions on the cheap end, one-off luxury features on the expensive end).

What SHAP Showed Me

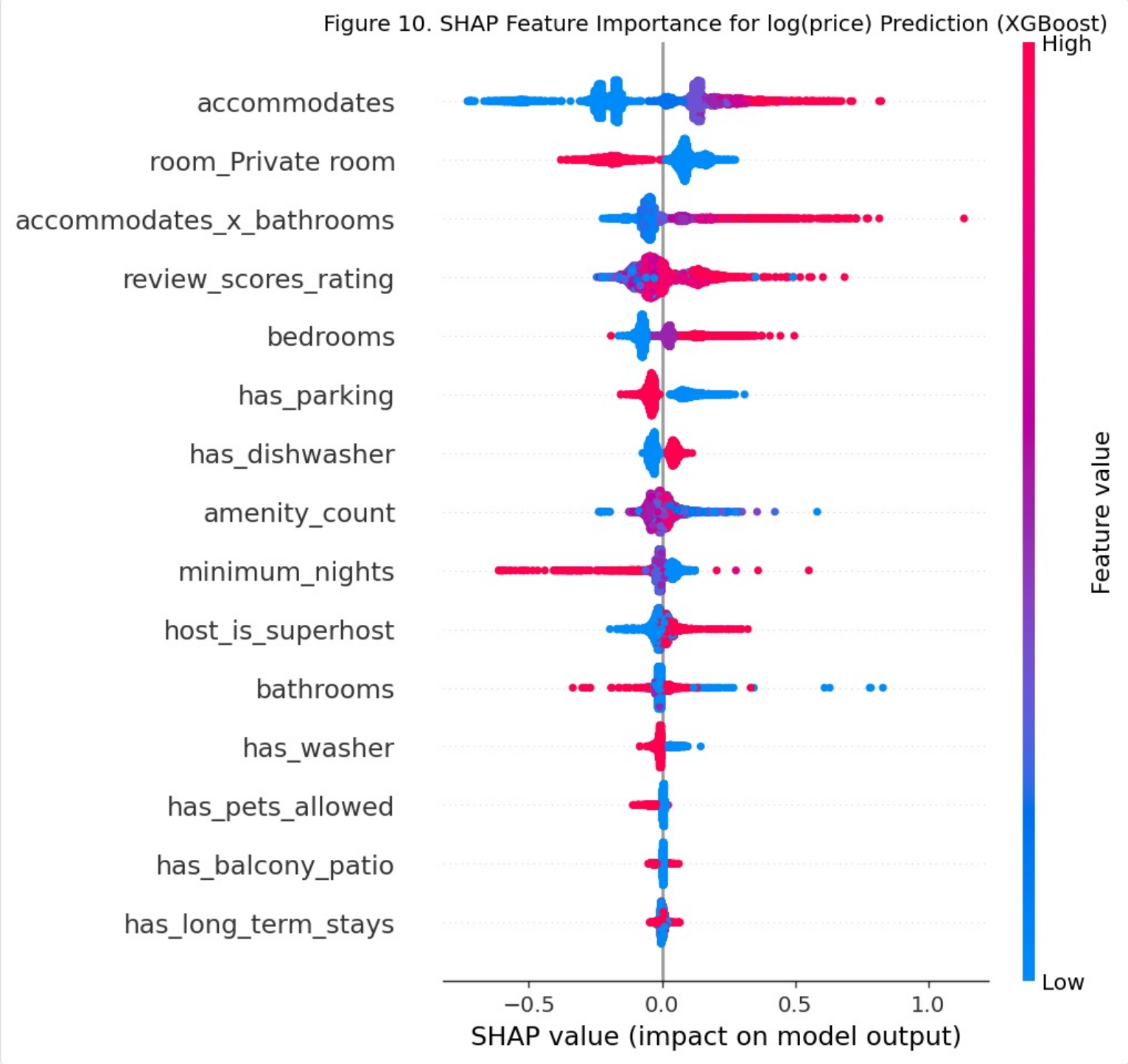


Figure 10. SHAP feature importance for log(price) prediction (XGBoost). Red = high feature value, blue = low.

`accommodates` dominates everything else, which lines up with both the regression in 7.3 and the correlation work in Section 6 (Spearman ρ=0.675). After that: whether it's a private room, the accommodates×bathrooms interaction, review score, and bedroom count, all consistent with what the regression already told me.

THE ONE RESULT THAT GENUINELY SURPRISED ME

SHAP showed having parking associated with *lower* predicted price, which seemed backwards, so I dug into it rather than just writing it down. Checking parking by neighbourhood: it's near 100% in outer areas like Parkhead and Sighthill, South Gyle, West Pilton, and only 47.5% in Old Town, Princes Street and Leith Street, the same neighbourhood that came out most expensive back in Section 6.3. Parking isn't a desirable amenity here so much as a marker of being outside the dense, pricier centre, where there's simply no room for it and most guests don't need it anyway. I wouldn't tell a host to remove parking to raise their price. That's reading causation into something that's really just correlation, and this is a pretty clean example of why that distinction matters.

What I'd Do Differently With More Time

- A MAPE around 33% means predictions are typically off by roughly a third. Usable, not precise. That tracks with everything else in this report pointing at roughly half of price variance being driven by things I haven't measured.
- I never touched the listing description text. The free-text fields are rich (lots of tourist-destination language, per the Day 1 profiling) and could plausibly add real signal. A natural next step.

- Small categories like Shared room and Hotel room have unstable error estimates simply because there isn't much data. Any real deployment of this model should flag predictions in those categories as lower confidence rather than presenting them with the same apparent precision as the bigger categories.

09 AI/ML Experiments

I made a deliberate call to skip Section 7 of the brief — the NLP-on-reviews, LLM-insight-generation, recommendation-system, and agentic work — in favor of going deeper on the price-prediction work in Section 8. See Section 2 for the full reasoning and Section 14 for what I'd actually build here given more time.

> **AI TOOLS IN PRODUCING THIS REPORT ITSELF**
>
> While I didn't pursue Section 7's specific analytical experiments, I did use Claude throughout the engineering and writing process for this assignment — for drafting code, debugging, and structuring this report. The full breakdown of how, including what got validated and where I rejected or rewrote AI suggestions, is in Appendix A.

What I'd Build Next

The reviews table is already sitting in the dimensional model, joined to `fact_listing` by listing ID, so the obvious next step is sentiment analysis on the 559,087 reviews, checked against the numerical review score already used throughout Sections 6 through 8. I'd want to know whether review text adds predictive signal the score alone misses, and whether it could help with the price-prediction model's weak spots at the extremes (Section 8). A natural second step would be a Q&A interface over the reviews corpus, so a stakeholder could ask questions directly instead of reading a static report.

10 Visualizations Index

Figure	Title	Section
1–2	Distribution of nightly price; price by room type	6.1
3–4	Median price by neighbourhood; listing density by neighbourhood	6.3
5	Availability trend (daily + 7-day rolling average) and by day of week	6.4
6–7	Host portfolio size distribution; market concentration curve	6.2
8–9	Review score distribution; review count vs. price	6.5
10	SHAP feature importance for log(price) prediction	8

Every figure here came straight out of the star schema described in Section 5, generated by the scripts in `src/` and walked through in the fully-executed notebook `notebooks/02_eda_and_statistics.ipynb`. Source PNGs are in `notebooks/figures/`.

11 Business Recommendations

For Hosts

- Property type and guest capacity are the biggest levers you actually have (Section 7.3, Section 8). Converting a private room to a whole-property listing, or accommodating more guests where it's feasible, moves price more than any single amenity will.
- Don't add parking expecting it to raise your price. The relationship runs the other way around: it's a location effect, not a cause (Section 8). If you're in a central, parking-scarce area, that's not a gap to fix.
- Charging a bit more for weekends is reasonable, but keep it modest. The weekend demand effect is real and consistent, but small (Section 7.1, H5).

For Platform Operators or Market Analysts

- Edinburgh's host base genuinely splits into two groups (Section 6.2): 79.4% single-listing hosts alongside a top-10% cohort controlling 38% of supply. Policy and support decisions probably shouldn't assume one typical host.
- Review score alone isn't a strong way to judge listing quality right now. 91.3% of listings sit at 4.5+ stars (Section 6.5). Any internal ranking system should weight review count, recency, and sub-scores alongside the headline rating, not the rating by itself.
- Closing the calendar pricing gap should be a real priority for future data collection (Section 13). Its absence limited how precisely I could speak to seasonal and weekend pricing throughout this report.

For Investors

- The airport-adjacent cluster (Dalmeny, Kirkliston and Newbridge, Section 6.3) is a genuinely different, less crowded niche serving larger travel groups, separate from the high-competition city-centre market.
- A pricing model built purely on listing characteristics gets you to roughly 33% mean error (Section 8). That's a reasonable starting estimate, not something I'd treat as a precise valuation.

12 Cross-City Comparisons

Not applicable here. As I explained in Section 2, this is a single-city analysis by choice, and the assignment itself treats that as a legitimate, well-regarded option distinct from multi-city work, which is really meant to demonstrate scalability and automation specifically.

That said, the pipeline wasn't built with blinders on. City configuration lives in one file (`src/config.py`), and the cleaning and profiling functions branch on file type rather than hardcoding anything Edinburgh-specific, so extending this to more cities wouldn't mean rewriting the pipeline. See Section 14 for what that extension would actually involve.

13 Limitations & Caveats

About the Data

- **The calendar pricing gap.** Confirmed as a genuine source limitation, not something I broke. It directly limited how precisely H5 (Section 7.1) and the seasonal analysis (Section 6.4) could speak to actual pricing rather than just availability.
- **One snapshot in time.** This is a scrape from 21 September 2025. It can't show listings added or removed outside that date, and it has no access to real booking transactions. Airbnb doesn't publish that data, so review counts stand in as a demand proxy throughout, which is an imperfect but widely-used substitute.
- **Small categories.** Hotel room (n=21) and Shared room (n=19) just don't have enough listings for stable statistics. Anywhere these show up, I've flagged the sample size rather than presenting the numbers with more confidence than they deserve.

About the Models

- Both the regression (R^2=0.414) and the ML model (R^2≈0.49) leave roughly half of price variance unexplained. Likely culprits include exact micro-location, photo quality, listing description quality, and host responsiveness, none of which were available in usable structured form here.
- Regression residuals stay right-skewed and heavy-tailed even after log-transforming price, which means a handful of outlier listings are probably still pulling on the fit more than I'd like.
- The ML model has a clear U-shaped error pattern by price range (Section 8), weakest at both the cheap and very expensive ends, where there's just less data to learn from.

About Scope

As covered in Section 2, I skipped Section 3.6 (cloud-native topics), Section 5.4 (multi-city), and most of Section 7 (NLP/LLM/agentic work beyond what's disclosed in Section 9). That was a deliberate choice in favor of depth elsewhere, not something that fell through the cracks.

14 Future Improvements

Given More Time

- Sentiment analysis on the 559,087 reviews (Section 9). Does the text add anything the numerical score misses, and could it help close the error gap at the price extremes from Section 8?
- Pull features out of the listing description and neighbourhood overview text (basic TF-IDF or embeddings) to see if they explain any of the roughly 50% of price variance that's currently unaccounted for.
- Extend to 2–3 more UK cities. The pipeline's config-driven design (Section 5) was built with this in mind. It would also let me check whether the parking/location finding from Section 8 and the festival seasonality pattern from Section 6.4 actually generalize, or whether they're specific to Edinburgh's particular geography and tourism calendar.

Given Better Data

- If a future scrape actually has calendar-level pricing, I could test weekend and seasonal price premiums directly instead of relying on the availability proxy used throughout Section 7 and Section 6.4.
- Real booking data, if it were ever available, would replace review count as a demand proxy with something far more direct, and would probably change some of the conclusions in H3 (Section 7.1) and the review-count analysis in Section 6.5.

Given More Resources

The production-readiness work scoped out in Section 5: containerizing the pipeline, a real orchestration layer instead of running scripts in sequence, automated data-quality checks as code rather than the manual profiling I did, and proper lineage tracking.

15 Reflection

How I Prioritized

I tried to let the rubric's actual weights drive where my time went, rather than spreading effort evenly across every section. Problem Solving and Data Engineering Quality carry the most weight, so the mandatory and recommended sections got the bulk of my time. For the optional work, I picked one thing (price prediction) and did it properly instead of touching three things superficially.

What I Traded Off

The biggest one: skipping Section 7's NLP/LLM work and Section 3.6's cloud-native topics entirely to go deeper on price prediction. The brief itself says a candidate who nails two sections beats one who half-finishes everything, and I took that seriously rather than treating it as a throwaway line.

What I'd Actually Want Someone to Take Away From This

If I'm honest, the most useful thing I demonstrated here isn't any individual chart or number. It's the habit of checking my own work before trusting it — running a second calculation against a counter-intuitive result, re-checking a formula against its documentation when something looked off, actually executing every script and notebook instead of assuming code that runs without crashing is necessarily correct. I caught a handful of real mistakes this way during this project: a VIF calculation that was wrong because of a dropped constant term, a boolean comparison bug from a wrong assumption about how DuckDB stores types, a notebook that failed on first run because of one missing import. All of them are written up plainly in the decision log rather than quietly fixed and hidden. That habit matters more to me, long-term, than any single result in this report.

A Appendix A — AI Usage Disclosure

Tools Used

Claude (Anthropic), through the Claude.ai interface, throughout the engineering and writing process.

Where AI Was Involved

Component	How AI was used
Pipeline code (`src/*.py`)	Drafted with AI assistance, based on my own decisions about approach (the star schema design, test selection logic, and so on). I reviewed and validated every module by actually running it against the real dataset
Statistical test selection & effect sizes	AI helped me research appropriate non-parametric alternatives once I'd confirmed assumption violations in the data; I checked the actual test choices against the real Shapiro-Wilk and Levene results from this dataset rather than taking a suggestion on faith
This report's writing	Drafted with AI assistance; every number and claim was checked against the underlying JSON and CSV output before it went in
Planning and prioritization	AI helped me map available time against the rubric's stated weights to decide where to go deep

How I Validated Output

I ran every script in `src/` against the real dataset rather than trusting that written code would also be correct code. The full pipeline, from ingestion through ML, was re-run end to end from raw data several times during development to confirm it actually reproduces. The combined notebook was executed through `jupyter nbconvert --execute` and checked programmatically for error outputs across all 38 cells before I accepted it as finished.

Where I Caught and Fixed Mistakes

A few AI-assisted drafts had real errors in them that I only caught by checking against the actual data, not by trusting the output at face value:

- A VIF calculation initially reported implausibly high values (15.2, 9.1), caused by dropping the regression constant before calling statsmodels' VIF function. I noticed the raw correlations between predictors didn't support numbers that high, which sent me back to check the actual usage pattern in the documentation.
- One statistical test (H5) produced a meaningless result (p=1, with a divide-by-zero warning) because I'd assumed DuckDB had kept a column as raw text when it had actually inferred it as a native boolean during ingestion. Caught by checking the column's real stored type directly rather than assuming the original CSV format had survived.
- An early neighbourhood price map and an early seasonal chart both initially implied stronger conclusions than the data actually supported — a 2-listing neighbourhood read as "most expensive," a single festival-month dip presented as the whole seasonal story when a separate weekly cycle was actually the bigger pattern. Both were caught by manually checking the numbers behind the chart before trusting it.
- A notebook cell failed the first time I ran it because of a missing `import shap` line, caught simply because I actually executed the notebook instead of assuming a cell that looked right would also run right.

The full, dated record of every decision — including all of the above — is in `reports/00_assumptions_and_decisions.md` (24 entries). I'd consider that log part of what I'm actually submitting here, not a footnote.

Changes I Made to AI-Drafted Content

Beyond the fixes above, I rewrote several AI-drafted business interpretations where the language overstated how meaningful a result actually was relative to its statistical significance. The clearest example is the H5 result in Section 7.1, where an early draft called the effect "modest" — I changed that to "negligible by convention," which is what the actual Cramer's V value supports.